

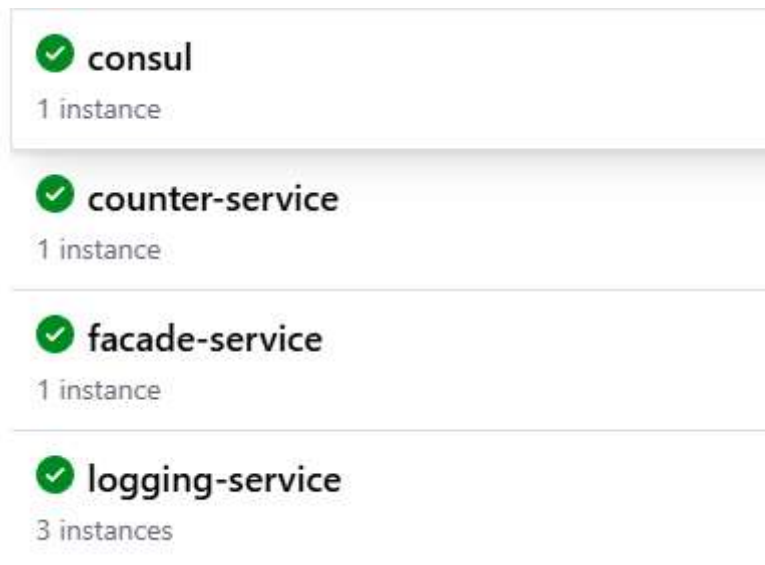
Task 5 — Microservices with Consul

GitHub: https://github.com/Romko-studik/apz_homework_2/tree/micro_consul

1. Service Registration in Consul

All services register themselves dynamically on startup

Consul UI — all registered services:



Consul API — service registry:

```
PS D:\UNI_UCU\APZ\apz_homework_2> Invoke-RestMethod -Uri "http://localhost:8500/v1/agent/services"

counter-service-1 : @({ID=counter-service-1; Service=counter-service; Tags=System.Object[]; Meta=; Port=8002; Address=counter-service; Weights=; EnableTagOverride=False; Datacenter=dc1}
facade-service-1 : @({ID=facade-service-1; Service=facade-service; Tags=System.Object[]; Meta=; Port=8000; Address=facade-service; Weights=; EnableTagOverride=False; Datacenter=dc1}
logging-service-1 : @({ID=logging-service-1; Service=logging-service; Tags=System.Object[]; Meta=; Port=8001; Address=logging-service-1; Weights=; EnableTagOverride=False; Datacenter=dc1}
logging-service-2 : @({ID=logging-service-2; Service=logging-service; Tags=System.Object[]; Meta=; Port=8001; Address=logging-service-2; Weights=; EnableTagOverride=False; Datacenter=dc1}
logging-service-3 : @({ID=logging-service-3; Service=logging-service; Tags=System.Object[]; Meta=; Port=8001; Address=logging-service-3; Weights=; EnableTagOverride=False; Datacenter=dc1}
```

2. Consul KV Config

Hazelcast and MQ configuration stored in Consul Key/Value store

Consul UI — Key/Value store:

Key / Value

Type ▾

Name


hazelcast


mq

3. Fault Tolerance — instance down

Stopping logging-service-1 and showing Consul detects it and redirects calls

Consul UI — logging-service-1 unhealthy:

logging-service-1	✖ All service checks failing ✔ All node checks passing 843bbd0453e2 🌐 logging-service-1:8001
logging-service-2	✔ All service checks passing ✔ All node checks passing 843bbd0453e2 🌐 logging-service-2:8001
logging-service-3	✔ All service checks passing ✔ All node checks passing 843bbd0453e2 🌐 logging-service-3:8001

Transaction still succeeds via remaining instances:

```
PS D:\UNI_UCU\APZ\apz_homework_2> Invoke-RestMethod -Method POST -Uri "http://localhost:8000/transaction" -ContentType "application/json" -Body '{"user_id": "test", "amount": 111}'
>> Invoke-RestMethod -Uri "http://localhost:8000/accounts" | ConvertTo-Json

transaction_id      status
-----
7a14061f-d034-420c-b47c-59c4c78adc84 queued
{
  "balances": {
    "user1": 300.0,
    "user2": 600.0,
    "user3": 900.0,
    "user4": 1200.0,
    "user5": 1500.0,
    "user6": 1800.0,
    "user7": 2100.0,
    "user8": 2400.0,
    "user9": 2700.0,
    "user10": 3000.0,
    "test": 111.0
  }
}
```

4. Performance Testing

Comparing results across Task 1, Task 3, and Task 5

Scenario 1 — 10 clients x 10K transactions on own accounts:

```

PS D:\UNI_UCU\APZ\apz_homework_2> py .\client\tests.py --scenario 1

Scenario 1 — 10 clients x 10,000 transactions
Total time : 669.93s
Req/s      : 149.3
Logging network avg : 31.21ms
Counter queue avg   : 31.21ms
Logging processing avg : 1.8927ms

Correctness check:
user_0: 10000.0 CORRECT
user_1: 10000.0 CORRECT
user_2: 10000.0 CORRECT
user_3: 10000.0 CORRECT
user_4: 10000.0 CORRECT
user_5: 10000.0 CORRECT
user_6: 10000.0 CORRECT
user_7: 10000.0 CORRECT
user_8: 10000.0 CORRECT
user_9: 10000.0 CORRECT

```

Scenario 2 — 10 clients x 10K transactions on shared account:

```

PS D:\UNI_UCU\APZ\apz_homework_2> py .\client\tests.py --scenario 2

Scenario 2 — 10 clients x 10,000 transactions
Total time : 667.51s
Req/s      : 149.8
Logging network avg : 30.97ms
Counter queue avg   : 30.97ms
Logging processing avg : 1.8335ms

Correctness check:
shared_user: 100000.0 CORRECT

```

Metric	Task 1 (in-memory)	Task 3 (DB + HZ)	Task 5 (Consul)
10 accounts — Total time	535s	556s	670s
10 accounts — Req/s	187	180	149
10 accounts — Logging avg	5.37ms	6.77ms	31.21ms
10 accounts — Counter avg	5.20ms	6.39ms	31.21ms
1 account — Total time	545s	560s	668s
1 account — Req/s	183	179	150
1 account — Logging avg	5.48ms	7.12ms	30.97ms
1 account — Counter avg	5.66ms	6.45ms	30.97ms

Analysis

Overall even though the completion time is longer in the latter tasks the overall implementations are still much better. We have to consider that for each request the number of services which are used is increased, which also increases the overall time. However, this massively increases the systems resilience and overall functionality - for the program from the first task even one service down is fatal. On the other hand we have safeguards for each service in the latest implementation, which is a great win considering only 15% slowdown.